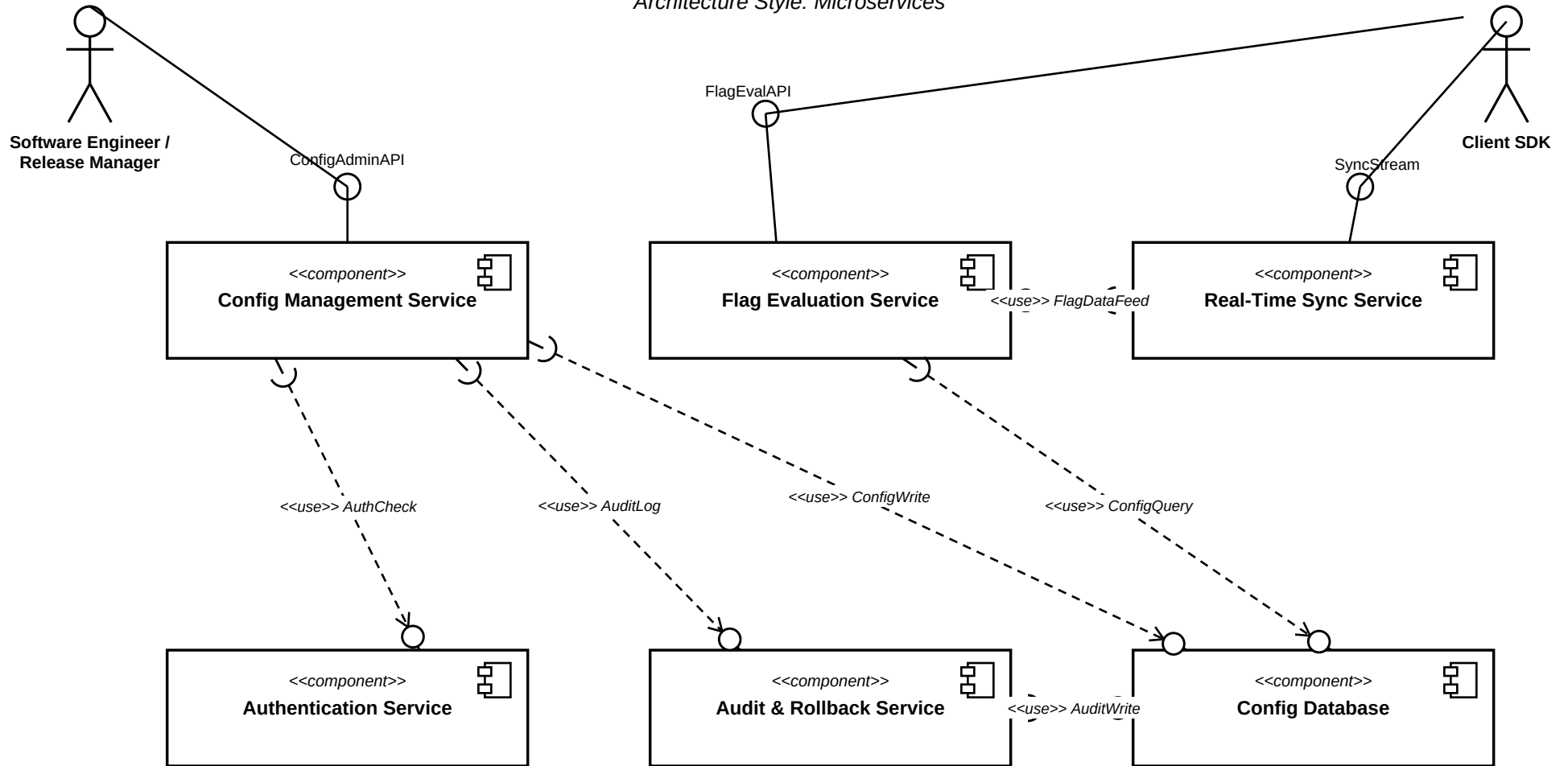


UML Component Diagram: Feature Flag & Dynamic Config Manager

Architecture Style: Microservices



Architecture Selection & Justification

Problem Statement #43 — Feature Flag & Dynamic Config Manager

We chose Microservices Architecture for the Feature Flag & Dynamic Config Manager System.

Two Scenario-Related Reasons

- Independent scaling: Flag evaluation is invoked on the hot path of every client-side feature check and receives orders of magnitude more traffic than administrative actions such as creating a flag or viewing the audit log. As a microservice, the Flag Evaluation Service and Real-Time Sync Service can be scaled horizontally (more instances, in-memory caching, read replicas) independently of the lower-traffic Config Management, Authentication, and Audit & Rollback services, avoiding the cost of over-provisioning the entire system to meet peak evaluation load.
- Independent deployment and fault isolation: Config changes, SDK synchronization, authentication, and audit logging evolve and fail at different rates. Deploying the Real-Time Sync Service for a protocol upgrade, or patching the Authentication Service, does not require redeploying or risking downtime in the Flag Evaluation Service. If the Audit & Rollback Service is temporarily degraded, flag evaluation for connected SDKs is unaffected, which is critical since the platform sits on other teams' request paths.

Security Advantage

Isolating Authentication as its own microservice creates a single, centrally enforced and centrally audited authorization boundary: every write path (Config Management, and indirectly Audit & Rollback) must call the AuthCheck interface before a flag or override can be modified, rather than each service re-implementing its own ad-hoc access checks. This reduces the attack surface for unauthorized cohort or rollout manipulation and makes it straightforward to rotate credentials or tighten policy in one place without touching the other services.

Performance Benefit

Because the Flag Evaluation Service is deployed and scaled separately from the heavier, write-oriented Config Management and Audit & Rollback services, it can be optimized purely for fast reads — e.g., in-memory/cached configuration and horizontally replicated instances placed close to client SDKs — keeping p99 evaluation latency under the 15 ms target (NFR-001) even while config writes, audits, or rollbacks are happening concurrently elsewhere in the system.